NODE.JS — LECTURE 2

Detailed Study Notes

Based on the uploaded lecture transcript • Organized for revision and GitHub

Main topics: npm and project setup, package.json vs package-lock.json, Nodemon, JavaScript functions and callbacks, Node.js built-in modules (os and fs), importing/exporting between files, and using Lodash.

Note: The source was an automatic speech-to-text transcript, so some technical names were misheard. Obvious terms have been normalized (for example, npm, package.json, package-lock.json, Nodemon, callback, fs, os, module.exports, and Lodash). Examples below are written as readable JavaScript.

1. What this lecture covers

The teacher continues the Node.js/backend journey from Lecture 1. The focus moves from running basic JavaScript files to setting up a Node project, installing packages, using callbacks, working with built-in Node modules, sharing code between files, and using a utility library.

- Create a Node.js project with **npm init**.
- Understand **package.json**, **package-lock.json**, and **node_modules**.
- Install and run **Nodemon** so code restarts after changes.
- Write functions in different JavaScript styles.
- Understand a **callback function** and when it is called.
- Use Node's built-in **os** and **fs** modules.
- Export values/functions from one file and use them in another.
- Use **Lodash** utilities for common data-handling tasks.

2. Check Node.js and create a project

Check that Node.js is installed

Open a terminal and run:

```
node -v
```

The command prints the installed Node.js version. If the command is not recognized or an error appears, Node.js may not be installed correctly or may not be available in PATH. The previous lecture covered installation.

Create and open a project folder

The teacher creates a folder for the tutorial code, opens that folder in VS Code, and uses the integrated terminal from inside that folder. A terminal command such as **mkdir** creates a directory; **cd** changes into a directory.

```
mkdir node-tutorial
cd node-tutorial
```

Folder names are examples; use your own project folder name. Make sure the terminal is currently inside the project folder before running npm commands.

3. npm and npm init

What is npm?

npm is the Node.js package manager. It provides access to packages/libraries created by the JavaScript community and lets you install them into a project. The teacher compares npm to a large shop: you choose the package you need, then install it into your project.

For example, **Express** is a package commonly used to build web servers. Nodemon is a development tool that watches files and restarts a Node process when files change.

Initialize a Node project

```
npm init
```

The command asks a series of questions, such as package name, version, description, entry point, author, and license. Press Enter to accept defaults when you do not need to change them. At the end, npm creates **package.json**.

The teacher describes this as filling in a basic project form: it records information about the project and can also contain scripts and dependency information.

What is package.json?

package.json is the project's manifest. It records project metadata and, among other things, the packages the project depends on and the scripts that can be run.

```
{
  "name": "node-tutorial",
  "version": "1.0.0",
  "description": "Node.js tutorial",
  "main": "server.js",
  "scripts": {},
  "author": "",
  "license": "ISC"
}
```

The exact fields and values depend on the answers you enter. The example is illustrative; it is not meant to reproduce the teacher's personal metadata.

Install a package

```
npm install express
```

This installs Express into the current project. The package is downloaded and placed in **node_modules**; npm also records dependency information in the project's package files.

The transcript also shows installing Nodemon for development. A common local-development install is:

```
npm install --save-dev nodemon
```

You may also see the shorter equivalent **npm i** for **npm install**. The important point is to run the command in the intended project directory.

4. package.json vs package-lock.json vs node_modules

Item	What it does	Easy way to remember
package.json	Project metadata, scripts, and declared dependencies (including version ranges).	The project description and dependency wish-list.
package-lock.json	Records the resolved dependency tree and specific installed versions, including nested dependencies, so installs can be reproduced more consistently.	The detailed record of what npm resolved.
node_modules/	Contains the package code installed for this project, including dependencies of dependencies.	The actual downloaded packages.

The teacher uses a shopping-list and receipt analogy: package.json is like the list of items you intend to buy; package-lock.json is like the detailed receipt showing the exact items/versions; node_modules contains the installed packages.

Why the lock file matters: a team may have several developers working on the same project. Keeping the lock file in version control helps npm install the same resolved dependency versions across machines and reduces “works on my machine” differences.

Do not edit node_modules by hand. If a package needs to be added or updated, use npm commands and review the package files.

5. Running a JavaScript file and using Nodemon

Run a file with Node.js

The teacher creates a JavaScript file such as **server.js** and runs it from the terminal.

```
// server.js
console.log("Server file is running");
```

```
node server.js
```

Node executes the JavaScript file and prints the message. After editing a file, save it and run the command again to see the new output.

Why Nodemon is useful

Repeatedly stopping and re-running the same command during development is inconvenient. **Nodemon** watches project files and restarts the Node process when it detects a change.

```
npm install --save-dev nodemon
```

```
npx nodemon server.js
```

The transcript demonstrates the idea that after a file is changed and saved, Nodemon restarts the script automatically. If Nodemon does not start, check that installation completed and that the command is run from the project directory.

Nodemon is a development convenience; it is not what makes Node.js run JavaScript. Node can run the file directly with **node server.js**.

6. JavaScript functions: several ways to write them

The teacher revises functions by using an addition example. A function accepts inputs (parameters), performs work, and can return a result.

Function declaration

```
function add(a, b) {
  return a + b;
}

const result = add(2, 3);
console.log(result); // 5
```

Function expression

```
const add = function (a, b) {
  return a + b;
};

console.log(add(2, 3)); // 5
```

Arrow function with a block body

```
const add = (a, b) => {
  return a + b;
};

console.log(add(2, 3)); // 5
```

Short arrow function

```
const add = (a, b) => a + b;

console.log(add(2, 3)); // 5
```

An arrow function with one expression can omit the braces and the explicit **return**; the expression's value is returned automatically. If you use braces, write **return** when you need to return a value.

Calling a function vs referring to a function

Code	Meaning
add	Refers to the function value; it does not call it.
add(2, 3)	Calls the function with arguments 2 and 3.

The teacher points out that parentheses after a function name generally indicate a function call. This is a useful beginner clue, although JavaScript has other syntax details that can affect how expressions are interpreted.

7. Callback functions

Definition

A **callback is a function passed to another function as an argument, so the receiving function can call it when appropriate**. The callback's name is not special; it is a normal function being used in a particular role.

The teacher explains the idea with an everyday sequence: first complete the main task, then do the next task after it finishes. The second action is like the callback.

Simple callback example

```
function add(a, b, callback) {
  const result = a + b;
  console.log("Result:", result);
  callback();
}

function done() {
  console.log("Addition completed");
}

add(3, 4, done);
```

Output:

```
Result: 7
Addition completed
```

How it works:

- The call **add(3, 4, done)** passes two numbers and the function **done**.
- Inside add, the result is calculated and printed.
- Then **callback()** calls the function that was passed in; in this example, that function is done.
- Because the main function controls when it calls callback, the callback runs at that point in the flow.

Pass the callback inline

```
function add(a, b, callback) {
  const result = a + b;
  console.log("Result:", result);
  callback();
}

add(2, 3, function () {
  console.log("Addition completed");
});
```

The callback can be written directly where the function is called, rather than defined separately.

Use an arrow function as the callback

```
add(2, 3, () => {
  console.log("Addition completed");
});
```

This works because the third argument is still a function. The shorter syntax does not change the central idea.

Important distinction: passing vs calling

```
add(2, 3, done);   // pass the function
add(2, 3, done()); // calls done immediately, then passes its return value
```

For this callback example, pass **done**, not **done()**. The receiving function is expected to call the callback later. Passing done() would run it immediately and would pass the result of that call instead.

The transcript's file-writing example also uses a callback: the callback handles what should happen after the file operation completes. In Node.js, many filesystem operations use callbacks to report completion or errors.

8. Node.js built-in modules

Node.js provides built-in modules that can be used without downloading them from npm. The teacher demonstrates the **os** and **fs** modules.

The os module

The **os** module provides information about the operating system and the current machine/user environment. Import it with CommonJS require:

```
const os = require("os");

const user = os.userInfo();
console.log(user);
console.log(user.username);
```

The exact object fields and values depend on the computer and operating system. The transcript demonstrates inspecting the returned object and printing the username.

The fs (file system) module

The **fs** module provides functions for working with files and directories: reading files, writing files, appending content, checking paths, and more.

```
const fs = require("fs");

console.log(fs); // inspect the module's available properties/functions
```

Printing the whole module can show a large object of functions. In everyday code, import fs and call the specific operation you need rather than printing the whole module.

Write or append a file with a callback

The transcript shows using a file-system function to write a greeting into a text file and to use a callback after the operation. A typical modern example is:

```
const fs = require("fs");

fs.appendFile("greeting.txt", "Hello, friends!\n", (err) => {
  if (err) {
    console.error("Could not write the file:", err);
    return;
  }

  console.log("Greeting saved");
});
```

appendFile appends the text to the file and creates the file if it does not exist. The callback receives an error value if the operation fails. If the goal is to replace the file's contents rather than append, use **fs.writeFile** instead.

The lecture's key learning point is how the filesystem function accepts the file path, data, and callback, and how the callback runs after the operation finishes. The callback should check the error argument.

9. Importing and exporting your own files (CommonJS)

The teacher demonstrates splitting code across files. One file can contain helper functions or data, and another file can use them. With CommonJS, the usual mechanism is **module.exports** to export and **require()** to import.

Example: export a value and a function

```
// notes.js
const age = 25;

function addNumbers(a, b) {
  return a + b;
}

module.exports = {
  age,
  addNumbers
};
```

Import and use them in another file

```
// server.js
const notes = require("./notes");

console.log(notes.age);                 // 25
console.log(notes.addNumbers(24, 18));  // 42
```

The relative path **./notes** points to the notes.js file in the same directory. Node loads that module and returns the value assigned to module.exports.

Common mistake: export what you want to use

A variable or function declared inside notes.js does not automatically become available in server.js. It must be exported, and the importing file must access the exported property or destructure it.

```
// Alternative import style
const { age, addNumbers } = require("./notes");

console.log(age);
console.log(addNumbers(24, 18));
```

The lecture uses CommonJS syntax. Modern Node.js projects can also use ES modules (**export**/**import**), but that is a separate module system; do not mix the syntaxes casually in one beginner example.

10. Using a third-party package: Lodash

The teacher introduces **Lodash**, a utility library with functions that make common data operations easier. The lecture's examples focus on extracting unique values from an array and checking whether a value is a string.

Install and import Lodash

```
npm install lodash
```

```
const _ = require("lodash");
```

The underscore (_) is a conventional variable name used for Lodash in many examples. It is not a JavaScript keyword; the library could technically be assigned to another variable, but following common conventions makes code easier for other developers to read.

Get unique values

```
const data = [1, 2, 2, 1, 3, 3];
const uniqueData = _.uniq(data);

console.log(uniqueData); // [1, 2, 3]
```

_.uniq(array) returns an array with duplicate values removed, preserving the first occurrence order.

Check whether a value is a string

```
console.log(_.isString("hello")); // true
console.log(_.isString(3));       // false
console.log(_.isString(true));    // false
```

_.isString(value) checks whether a value is a string. The result is a boolean: **true** or **false**. A string such as "3" is different from the number 3.

Why use a library?

The teacher's point is that developers do not need to manually reimplement every common data operation. Utility libraries provide tested, reusable functions. Still, learn what each function does and consult the official documentation; installing a package does not mean every function is automatically appropriate for every problem.

11. Important commands and concepts — quick reference

Command / concept	Purpose
node -v	Check the installed Node.js version.
npm init	Create a package.json interactively.
npm install express	Install Express into the project.
npm install --save-dev nodemon	Install Nodemon as a development dependency.
npx nodemon server.js	Run server.js with Nodemon watching for changes.
require("fs")	Load Node's built-in filesystem module using CommonJS.
require("os")	Load Node's built-in operating-system module.
module.exports = ...	Expose values/functions from a CommonJS module.
require("./notes")	Load a local module from a nearby file.
npm install lodash	Install the Lodash utility library.
_.uniq(array)	Remove duplicate values from an array.
_.isString(value)	Check whether a value is a string.

12. Common mistakes to avoid

- Running npm commands from the wrong directory; the package files may be created in the wrong place.
- Thinking package.json and package-lock.json are the same. They have related but different jobs.
- Expecting Node to restart a changed file automatically without using a watcher such as Nodemon or restarting the process yourself.
- Passing **callback()** when you mean to pass the function itself as **callback**.
- Forgetting that callback functions only run when the receiving code calls them.
- Expecting a variable or function from another file to be visible automatically without exporting it.
- Using the wrong relative path when calling require, such as forgetting **./** for a nearby local file.
- Ignoring the error argument in an asynchronous filesystem callback.
- Assuming the username or other os-module values will be identical on every computer.
- Confusing the number 3 with the string "3" when using type-checking functions.

13. Practice tasks

The teacher asks learners to complete the assignment shared with the course and to ask questions when they get stuck. The transcript does not provide the exact assignment questions, so the following are practice exercises derived from the topics demonstrated in this lecture—not a claim that these are the teacher's exact assignment.

- Create a fresh Node project using npm init and inspect package.json.
- Install Express and Nodemon. Run a simple server.js file with Nodemon and verify that saving a change restarts it.
- Write add(a, b) using a function declaration, a function expression, and an arrow function.
- Write a function that accepts a callback and calls it after printing a result.
- Use the os module to print the current user information and username.
- Use fs.appendFile to add a line to a text file and handle any error in the callback.
- Export a constant and a function from notes.js, then import and use them in server.js.
- Install Lodash and use _.uniq on an array containing duplicates.
- Use _.isString with a string, number, and boolean, and note the output.

14. Final revision summary

- **npm** manages packages; **npm init** creates the project's package.json.
- **package.json** stores project metadata and declared dependencies; **package-lock.json** records the resolved dependency tree; **node_modules** contains installed package code.
- **Nodemon** restarts a Node process when watched files change during development.
- Functions can be written as declarations, expressions, or arrow functions.
- A **callback** is a function passed into another function and called by that function at the appropriate time.
- **os** and **fs** are built-in Node modules; they do not need to be installed from npm.
- **module.exports** and **require()** let CommonJS files share functions and values.
- **Lodash** provides utility functions such as **_.uniq** and **_.isString**.

Source and accuracy note

These notes are based on the Lecture 2 transcript supplied in this chat. Speech-to-text errors were normalized for readability, and code examples were formatted into valid, understandable JavaScript. Explanatory clarifications (such as checking filesystem callback errors and distinguishing appendFile from writeFile) are included to make the demonstrated concepts safer and easier to study. The exact assignment linked by the teacher was not present in the transcript, so its questions have not been invented.